

Computational Labs for Discrete Mathematics in Python and Julia

James Quinlan ¹

¹ Department of Computer Science, University of Southern Maine

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Submitted: 26 April 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

Discrete Math Labs is a collection of computer-based labs designed to reinforce key concepts from an undergraduate discrete mathematics course. The labs provide hands-on practice in applying mathematical ideas through computational examples. Students gain insight into theoretical material while developing programming skills. The collection comprises approximately 14 sequential labs spanning core topics in discrete mathematics, each accompanied by fully worked solutions in both Python and Julia.

The topics covered in these labs are standard within discrete mathematics texts (e.g., see (Epp, 2010; Rosen, 2019)), including:

- Number theory
- Set theory
- Recurrence relations
- Proof techniques (induction, cases, contrapositive, contradiction)
- Graphs (including trees, representations, and traversal)
- Algorithms and complexity
- Formal languages (regular expressions)

These labs bridge mathematical theory and practical computation, enabling students to develop intuition grounded in concrete examples. They complement traditional pencil-and-paper problem sets with interactive coding activities. All labs are designed for use in the Google Colab notebook environment and are available as open-source resources.

Statement of Need

Discrete mathematics is a crucial part of the computer science curriculum, as recognized by the Joint Task Force on Computer Science Curricula, a collaboration among the Association for Computing Machinery (ACM), the IEEE Computer Society, and the Association for the Advancement of Artificial Intelligence (AAAI) (Kumar & Raj, 2024). Discrete mathematics is a requirement in ABET accreditation criteria (ABET, 2026). In a survey of 199 Computer Science (BA/BS) programs, (Brodley et al., 2024) reported 99% require discrete mathematics. Moreover, this report showed that discrete mathematics is a prerequisite for nearly all core computer science courses, including data structures (73 programs), algorithms (117 programs), and databases (17 programs), as well as advanced topics such as parallel computing and machine learning (Brodley et al., 2024).

Despite its significance, students often find the subject challenging due to its abstract nature, therefore is a potential barrier to student progression. Traditional courses focus on formal proof and symbolic reasoning, often without including any computational work. As a result, students may not see how discrete mathematics relates to applications in computer science.

Resources that currently exist tend to be narrow. Many concentrate on specific topics like

logic or basic set operations. This lab series covers a wide range of core topics and links them to programming practice. This approach lets students test their ideas, spot patterns, and confront abstract concepts in a tangible way. Students can improve their coding skills while also enhancing their mathematical knowledge.

Traditional teaching methods depend on solving problems with pencil and paper. This method helps develop deductive reasoning but provides little chance for experimentation. There is a disconnect between mathematical theory and computational practice. These labs directly tackle that disconnect.

By integrating programming and data-driven exploration, students can:

- Actively test and explore mathematical ideas (McMaster et al., 2007)
- Develop intuition through computational experimentation (Liu & Castellana, 2021)
- Recognize the relevance of discrete structures in algorithms and software (Friend et al., 2023)
- Reinforce formal concepts through concrete examples (Raykova & Boev, 2018)

Analogous to laboratory experiences in the natural sciences, these computational labs provide a structured environment for experimentation in discrete mathematics. They support conceptual understanding while promoting engagement, confidence, and skill development in both mathematics and programming.

Course Description and Learning Objectives

This table maps the course learning objectives to the corresponding ABET student outcomes, demonstrating how the lab-based discrete mathematics course supports core computing competencies. The objectives emphasize applying mathematical concepts through programming, designing and analyzing computational solutions, and using experimentation to evaluate results. Strong alignment is evident with ABET outcomes 1 - 4, reflecting a focus on problem-solving, solution design, and evidence-based reasoning, while also incorporating communication skills through the interpretation and presentation of computational findings (ABET, 2026).

ABET Outcomes

1. Analyze complex computing problems and apply computing and mathematics
2. Design solutions to meet specified needs
3. Communicate effectively
4. Make informed judgments based on analysis and evaluation
5. Function effectively on a team

Course Learning Objective	ABET Outcome(s)
1. Apply discrete mathematical concepts using computational methods in Python and Julia.	1
2. Analyze mathematical structures and relationships through algorithmic implementations and experimental verification.	1, 4
3. Construct functions and programs that model discrete processes such as recursion, iteration, and combinatorial computation.	1, 2
4. Evaluate mathematical conjectures and identities using computational evidence and pattern recognition.	4
5. Interpret connections between discrete mathematics and core computer science topics such as algorithms, data structures, and networks.	1, 2

Course Learning Objective	ABET Outcome(s)
6. Design computational solutions that represent and manipulate abstract structures, including sets, graphs, and matrices.	2
7. Compare theoretical results with computational outcomes to understand correctness, efficiency, and growth.	1, 4
8. Develop problem-solving strategies integrating mathematical reasoning with programming-based experimentation.	2, 4
9. Communicate results and interpretations of computational experiments clearly in written or code-based form.	3, 4

78 The labs can be adapted for collaborative work to address ABET Outcome #5.

79 Lab Development

80 These labs were developed over several semesters for a four-credit undergraduate discrete
81 mathematics course with an integrated laboratory component. Students use the Google
82 Colab environment to avoid software installation and to support accessibility. The
83 notebooks support both Python and Julia, allowing flexibility in language choice while
84 maintaining a consistent computational framework.

85 Acknowledgements

86 We thank the teaching assistants and students for sharing feedback to improve the
87 laboratory assignments, including code debugging, clarification of problem statements,
88 and length.

89 References

- 90 ABET. (2026). *Criteria for Accrediting Computing Programs, 2025-2026*. <https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-computing-programs-2025-2026/>.
91
92
- 93 Brodley, C. E., Quam, M., & Weiss, M. (2024). An analysis of the math requirements
94 of 199 cs bs/ba degrees at 158 us universities. *Communications of the ACM*, 67(8),
95 122–131. <https://doi.org/10.1145/3661482>
- 96 Epp, S. S. (2010). *Discrete mathematics with applications*. Brooks/Cole Publishing Co.
- 97 Friend, M., Swift, A. W., Love, B., & Winter, V. (2023). A wolf in lamb's clothing:
98 Computer science in a mathematics course. *Proceedings of the 54th ACM Technical
99 Symposium on Computer Science Education v. 1*, 256–262. <https://doi.org/10.1145/323410.319070>
100
- 101 Kumar, A. N., & Raj, R. K. (2024). Computer science curricula 2023 (CS2023): The
102 final report. *Proceedings of the 55th ACM Technical Symposium on Computer Science
103 Education v. 2*, 1867–1868. <https://doi.org/10.1145/3626253.3633405>
- 104 Liu, Y. A., & Castellana, M. (2021). Discrete math with programming: A principled
105 approach. *Proceedings of the 52nd ACM Technical Symposium on Computer Science
106 Education*, 1156–1162. <https://doi.org/10.1145/3408877.3432537>
- 107 McMaster, K., Anderson, N., & Rague, B. (2007). Discrete math with programming:
108 Better together. *ACM SIGCSE Bulletin*, 39(1), 100–104. <https://doi.org/10.1145/1227504.1227348>
109

- 110 Raykova, M., & Boev, S. (2018). How to improve teaching in discrete mathematics via
111 programming and vice versa. *Computer Science and Education in Computer Science*,
112 211–228.
- 113 Rosen, K. H. (2019). *Discrete mathematics and its applications, seventh edition*. New
114 York, NY, USA: McGraw-Hill.

DRAFT